

Diagrama de clases y manual de arquitectura

Proyecto CRUD Preentrega Completa — Java Consola

1. Objetivo del documento

Este documento explica la arquitectura del proyecto de preentrega del CRUD de artículos y categorías desarrollado en Java para consola.

El objetivo es que el alumno pueda comprender no solamente qué clases existen, sino también por qué están organizadas de esa manera.

Se incluye:

- diagrama de clases
 - explicación de paquetes
 - explicación de responsabilidades
 - explicación de relaciones entre clases
 - arquitectura implementada
 - por qué no es un MVC puro
 - buenas prácticas aplicadas
 - conceptos de POO presentes en el proyecto
-

2. Diagrama de clases general

El siguiente diagrama representa las clases principales del proyecto y sus relaciones.

```
classDiagram
    class App {
        +main(String[] args) void
        -leerEntero(Scanner scanner, String mensaje) int
    }

    class Calculable {
        <<interface>>
        +calcularPrecioFinal() double
    }

    class Identificable {
        <<interface>>
        +getCodigo() int
    }
```

```

class Categoria {
    -int codigo
    -String nombre
    -String descripcion
    +Categoria(int codigo, String nombre, String descripcion)
    +getCodigo() int
    +setCodigo(int codigo) void
    +getNombre() String
    +setNombre(String nombre) void
    +getDescripcion() String
    +setDescripcion(String descripcion) void
    +toString() String
}

class Artículo {
    <<abstract>>
    #int codigo
    #String nombre
    #double precio
    #Categoria categoria
    +Articulo(int codigo, String nombre, double precio, Categoria
categoria)
    +getCodigo() int
    +setCodigo(int codigo) void
    +getNombre() String
    +setNombre(String nombre) void
    +getPrecio() double
    +setPrecio(double precio) void
    +getCategoria() Categoria
    +setCategoria(Categoria categoria) void
    +getTipoArticulo()* String
    #getDetalleEspecifico()* String
    +toString() String
}

class ArtículoElectronico {
    -int garantiaMeses
    +ArticuloElectronico(int codigo, String nombre, double precio,
Categoria categoria, int garantiaMeses)
    +getGarantiaMeses() int
    +setGarantiaMeses(int garantiaMeses) void
    +getTipoArticulo() String
    #getDetalleEspecifico() String
    +calcularPrecioFinal() double
}

class ArtículoAlimenticio {
    -int diasParaVencimiento
    +ArticuloAlimenticio(int codigo, String nombre, double precio,
Categoria categoria, int diasParaVencimiento)
    +getDiasParaVencimiento() int

```

```

        +setDiasParaVencimiento(int diasParaVencimiento) void
        +getTipoArticulo() String
        #getDetalleEspecifico() String
        +calcularPrecioFinal() double
    }

    class Repositorio~T extends Identificable~ {
        -ArrayList~T~ lista
        +Repositorio()
        +agregar(T objeto) boolean
        +listar() List~T~
        +buscarPorCodigo(int codigo) T
        +eliminar(T objeto) boolean
        +estaVacio() boolean
        +cantidad() int
    }

    class Menu {
        <<abstract>>
        #Scanner scanner
        +Menu(Scanner scanner)
        +mostrarMenu()* void
        +ejecutar()* void
        #leerEntero(String mensaje) int
        #leerDouble(String mensaje) double
        #leerTexto(String mensaje) String
        #leerSiNo(String mensaje) boolean
    }

    class MenuArticulos {
        -Repositorio~Articulo~ repositorioArticulos
        -Repositorio~Categoria~ repositorioCategorias
        +MenuArticulos(Scanner scanner, Repositorio~Articulo~
repositorioArticulos, Repositorio~Categoria~ repositorioCategorias)
        +mostrarMenu() void
        +ejecutar() void
        -ingresarArticulo() void
        -listarArticulos() void
        -consultarArticulo() void
        -modificarArticulo() void
        -eliminarArticulo() void
        -pedirNombreArticulo() String
        -pedirPrecioArticulo() double
        -pedirGarantia() int
        -pedirDiasParaVencimiento() int
        -pedirCategoriaExistente() Categoria
        -existeArticuloDuplicado(String nombre, Categoria categoria, int
codigoExcluir) boolean
    }

    class MenuCategorias {

```

```

-Repositorio~Categoria~ repositorioCategorias
-Repositorio~Articulo~ repositorioArticulos
+MenuCategorias(Scanner scanner, Repositorio~Categoria~
repositorioCategorias, Repositorio~Articulo~ repositorioArticulos)
+mostrarMenu() void
+ejecutar() void
-ingresarCategoria() void
-listarCategorias() void
-consultarCategoria() void
-modificarCategoria() void
-eliminarCategoria() void
-pedirNombreCategoria(int codigoExcluir) String
-pedirDescripcionCategoria() String
-existeNombreCategoria(String nombre, int codigoExcluir) boolean
-categoriaTieneArticulosAsociados(Categoria categoria) boolean
}

class Validaciones {
    <<utility>>
    -Validaciones()
    +validarTextoNoVacio(String texto) boolean
    +validarLongitudMaxima(String texto, int maximo) boolean
    +validarNoNegativo(int valor) boolean
    +validarNoNegativo(double valor) boolean
}

class Secuencias {
    <<utility>>
    -int proximoCodigoArticulo
    -int proximoCodigoCategoria
    -Secuencias()
    +generarCodigoArticulo() int
    +generarCodigoCategoria() int
}

Identificable <|.. Categoria
Identificable <|.. Articulo
Calculable <|.. Articulo

Articulo <|-- ArticuloElectronico
Articulo <|-- ArticuloAlimenticio

Articulo --> Categoria : tiene una

Menu <|-- MenuArticulos
Menu <|-- MenuCategorias

MenuArticulos --> Repositorio~Articulo~ : usa
MenuArticulos --> Repositorio~Categoria~ : usa
MenuCategorias --> Repositorio~Categoria~ : usa
MenuCategorias --> Repositorio~Articulo~ : usa

```

```
Repositorio --> Identificable : T debe implementar
```

```
App --> MenuArticulos : crea y ejecuta  
App --> MenuCategorias : crea y ejecuta  
App --> Repositorio~Articulo~ : crea  
App --> Repositorio~Categoria~ : crea
```

```
MenuArticulos ..> Validaciones : usa  
MenuCategorias ..> Validaciones : usa  
MenuArticulos ..> Secuencias : usa  
MenuCategorias ..> Secuencias : usa
```

3. Cómo leer el diagrama

Antes de analizar la arquitectura, conviene entender cómo se lee el diagrama.

Flecha de herencia

Cuando aparece:

```
Articulo <|-- ArticuloElectronico
```

significa:

```
ArticuloElectronico hereda de Articulo.
```

También podemos decir:

```
ArticuloElectronico es un Articulo.
```

Flecha de implementación

Cuando aparece:

```
Identificable <|.. Categoria
```

significa:

```
Categoria implementa la interfaz Identificable.
```

Es decir, `Categoria` se compromete a tener el método definido por esa interfaz.

Relación de uso

Cuando aparece:

```
MenuArticulos --> Repositorio<Articulo>
```

significa que `MenuArticulos` necesita un repositorio de artículos para funcionar.

No hereda de él.

No es un repositorio.

Lo usa.

Relación de composición simple

Cuando aparece:

```
Articulo --> Categoria : tiene una
```

significa que un artículo tiene una categoría asociada.

La relación correcta no es herencia, porque una categoría no es un artículo.

La relación correcta es composición, porque un artículo tiene una categoría.

4. Arquitectura implementada

El proyecto implementa una arquitectura de consola organizada por responsabilidades y paquetes.

No es una arquitectura MVC pura, porque no hay una vista separada formalmente ni controladores web.

Tampoco es una arquitectura por capas completa como la que se suele usar en Spring Boot.

Es una arquitectura didáctica, modular y progresiva para Java de consola.

Podemos describirla como:

```
Arquitectura modular por responsabilidades para aplicación de consola.
```

También puede entenderse como una primera aproximación a una arquitectura en capas:

```
App
↓
Menús
↓
Repositorios
↓
Modelos
```

Con clases utilitarias transversales:

```
Validaciones
Secuencias
```

5. Paquetes del proyecto

El proyecto está organizado en paquetes.

Un paquete en Java sirve para agrupar clases relacionadas.

La estructura es:

```
com.techlab.articulo
com.techlab.articulo.model
com.techlab.articulo.interfaces
com.techlab.articulo.repository
com.techlab.articulo.menu
com.techlab.articulo.utils
```

Esta organización evita tener todas las clases mezcladas en un solo lugar.

Además, ayuda a que el alumno empiece a pensar en responsabilidades.

6. Paquete principal

```
com.techlab.articulo
```

Contiene la clase principal:

App

App tiene el método:

```
public static void main(String[] args)
```

Ese método es el punto de entrada de la aplicación.

Cuando se ejecuta el programa, Java empieza por `main`.

7. Responsabilidad de App

La clase `App` no contiene toda la lógica del sistema.

Eso es una mejora importante.

Su responsabilidad principal es inicializar y coordinar.

En `App` se crean:

```
Scanner scanner = new Scanner(System.in);
```

```
Repositorio<Categoria> repositorioCategorias = new Repositorio<>();  
Repositorio<Articulo> repositorioArticulos = new Repositorio<>();
```

```
MenuCategorias menuCategorias = new MenuCategorias(...);  
MenuArticulos menuArticulos = new MenuArticulos(...);
```

Después, `App` muestra el menú principal y deriva la ejecución al menú correspondiente.

Si el usuario elige artículos:

```
menuArticulos.ejecutar();
```

Si el usuario elige categorías:

```
menuCategorias.ejecutar();
```

Esto permite que `App` no tenga que saber cómo se carga un artículo o cómo se elimina una categoría.

Solo coordina.

8. Por qué `App` no debería tener todo el código

En un proyecto principiante, es común poner todo en `main`.

Eso funciona al principio, pero rápidamente trae problemas.

Una clase gigante se vuelve difícil de:

- leer
- corregir
- probar
- mantener
- extender

Si mañana hay que modificar solo el CRUD de artículos, no conviene tocar una clase enorme con todo el sistema.

Por eso este proyecto separa la lógica en clases específicas.

Esta idea se relaciona con un principio muy importante de diseño:

Una clase debería tener una responsabilidad principal.

9. Paquete `model`

Responsabilidad

El paquete `model` contiene las clases que representan los objetos principales del sistema.

En este proyecto son:

Articulo
ArticuloElectronico
ArticuloAlimenticio
Categoria

Estas clases responden a la pregunta:

¿Qué cosas existen dentro del sistema?

Existen artículos.

Existen categorías.

Existen distintos tipos de artículos.

10. Clase `Categoria`

`Categoria` representa una categoría de artículos.

Tiene:

```
private int codigo;  
private String nombre;  
private String descripcion;
```

Esto significa que cada categoría guarda:

- un código identificador
- un nombre
- una descripción

Ejemplo conceptual:

```
Código: 1  
Nombre: Electrónica  
Descripción: Productos tecnológicos y electrónicos
```

`Categoria` implementa:

```
Identificable
```

Por eso debe tener:

```
public int getCodigo()
```

Esto permite que el repositorio genérico pueda buscar categorías por código.

11. Clase abstracta `Articulo`

`Articulo` es una clase abstracta.

Se declara así:

```
public abstract class Artículo implements Calculable, Identificable
```

Esta línea concentra varias ideas importantes.

abstract class Artículo

Significa que **Artículo** no puede instanciarse directamente.

No se puede hacer:

```
new Artículo(...)
```

¿Por qué?

Porque en el sistema no queremos artículos genéricos.

Queremos artículos concretos:

- electrónicos
- alimenticios

implements Calculable

Significa que todo artículo debe poder calcular su precio final.

implements Identificable

Significa que todo artículo debe tener un código identificador.

12. Atributos de **Artículo**

La clase **Artículo** contiene los datos comunes a todos los artículos:

```
protected int codigo;  
protected String nombre;  
protected double precio;  
protected Categoria categoria;
```

Estos atributos son comunes porque tanto un artículo electrónico como un artículo alimenticio tienen:

- código

- nombre
- precio
- categoría

La clase padre evita repetir esos atributos en cada clase hija.

13. Por qué los atributos de `Articulo` son `protected`

En este proyecto, los atributos de `Articulo` están declarados como `protected`.

Eso significa que pueden ser usados directamente dentro de la clase `Articulo` y también dentro de sus clases hijas.

Por ejemplo, `ArticuloElectronico` puede acceder a `precio` para calcular el precio final.

```
return precio * 1.10;
```

Para un proyecto didáctico, esto ayuda a ver claramente la herencia.

En un proyecto más estricto, también podría elegirse `private` y acceder mediante getters.

Ambas opciones son válidas, pero tienen distinto nivel de encapsulamiento.

14. Métodos abstractos de `Articulo`

`Articulo` tiene métodos abstractos:

```
public abstract String getTipoArticulo();
```

```
protected abstract String getDetalleEspecifico();
```

Un método abstracto no tiene cuerpo.

No tiene llaves con código.

Solo declara una obligación.

La clase padre dice:

Todo artículo debe poder informar su tipo.
Todo artículo debe poder informar su detalle específico.

Pero no define cómo.

Eso queda en manos de las clases hijas.

15. Clase `ArticuloElectronico`

`ArticuloElectronico` hereda de `Articulo`.

```
public class ArticuloElectronico extends Articulo
```

Esto significa:

Un artículo electrónico es un artículo.

Agrega un dato específico:

```
private int garantiaMeses;
```

Ese dato no está en todos los artículos.

Solo tiene sentido para artículos electrónicos.

16. Comportamiento de `ArticuloElectronico`

`ArticuloElectronico` implementa los métodos abstractos heredados.

Por ejemplo:

```
public String getTipoArticulo() {  
    return "Electrónico";  
}
```

Y:

```
protected String getDetalleEspecifico() {  
    return "garantía=" + garantiaMeses + " meses";  
}
```

También implementa el cálculo del precio final:

```
public double calcularPrecioFinal() {  
    if (garantiaMeses > 12) {  
        return precio * 1.10;  
    }  
    return precio;  
}
```

La regla es:

- si la garantía supera los 12 meses, se aplica un 10% de recargo
- si no, el precio final es el precio base

17. Clase `ArticuloAlimenticio`

`ArticuloAlimenticio` también hereda de `Articulo`.

```
public class ArticuloAlimenticio extends Articulo
```

Esto significa:

Un artículo alimenticio es un artículo.

Agrega un dato específico:

```
private int diasParaVencimiento;
```

Ese dato solo tiene sentido para productos alimenticios.

18. Comportamiento de `ArticuloAlimenticio`

`ArticuloAlimenticio` implementa:

```
public String getTipoArticulo() {  
    return "Alimenticio";  
}
```

Y:

```
protected String getDetalleEspecifico() {  
    return "días para vencimiento=" + diasParaVencimiento;  
}
```

También implementa:

```
public double calcularPrecioFinal()
```

con otra lógica.

La regla es:

- si vence en 3 días o menos, tiene 20% de descuento
- si vence en 7 días o menos, tiene 10% de descuento
- si no, conserva el precio base

Esto demuestra polimorfismo: el mismo método existe en distintos subtipos, pero se comporta diferente.

19. Paquete `interfaces`

El paquete `interfaces` contiene contratos.

En este proyecto hay dos interfaces:

```
Calculable  
Identificable
```

Una interfaz no representa necesariamente un objeto concreto.

Representa una capacidad o una obligación.

20. Interfaz `Calculable`

`Calculable` define:

```
double calcularPrecioFinal();
```

Esto significa:

Toda clase que implemente `Calculable` debe saber calcular un precio final.

En este proyecto, `Articulo` implementa `Calculable`.

Por lo tanto, todo artículo debe tener esa capacidad.

Las clases hijas concretas son las que definen la lógica real.

21. Interfaz `Identificable`

`Identificable` define:

```
int getCodigo();
```

Esto significa:

Toda clase que implemente `Identificable` debe poder devolver un código.

Esto es fundamental para el repositorio genérico.

El repositorio necesita poder buscar cualquier objeto por código.

Para eso necesita tener garantizado que el objeto tenga `getCodigo()`.

22. Paquete `repository`

El paquete `repository` contiene la clase:

```
Repositorio<T extends Identificable>
```

Esta clase administra objetos en memoria.

No usa base de datos.

Guarda los objetos en una lista:


```
private final ArrayList<T> lista;
```

23. Qué significa Repositorio<T extends Identificable>

Esta parte es clave.

T

representa un tipo genérico.

No es una clase real llamada T.

Es un espacio reservado para un tipo que se definirá después.

Ejemplo:

```
Repositorio<Articulo> repositorioArticulos = new Repositorio<>();
```

En ese caso, T pasa a ser Articulo.

Otro ejemplo:

```
Repositorio<Categoria> repositorioCategorias = new Repositorio<>();
```

En ese caso, T pasa a ser Categoria.

La parte:

```
extends Identificable
```

significa que T debe ser un tipo que implemente Identificable.

¿Por qué?

Porque el repositorio necesita hacer esto:

```
objeto.getCodigo()
```

Si `T` no tuviera esa restricción, Java no podría asegurar que el método `getCodigo()` existe.

24. Responsabilidad del repositorio

El repositorio se encarga de operaciones básicas sobre la lista:

```
agregar(T objeto)
listar()
buscarPorCodigo(int codigo)
eliminar(T objeto)
estaVacio()
cantidad()
```

Estas operaciones son comunes para artículos y categorías.

Sin repositorio genérico, habría que repetir lógica.

Por ejemplo, un método para buscar artículos por código y otro método muy parecido para buscar categorías por código.

El repositorio evita esa duplicación.

25. Por qué `listar()` devuelve una copia

El método `listar()` devuelve:

```
return new ArrayList<>(lista);
```

No devuelve la lista original.

Esto protege el encapsulamiento.

Si devolviera la lista original, otra clase podría modificarla directamente sin pasar por los métodos del repositorio.

Al devolver una copia, el repositorio mantiene más control sobre sus datos internos.

26. Paquete `menu`

El paquete `menu` contiene las clases relacionadas con la interacción por consola.

Incluye:

```
Menu
MenuArticulos
MenuCategorias
```

Estas clases se encargan de mostrar opciones, pedir datos al usuario y ejecutar acciones.

27. Clase abstracta `Menu`

`Menu` es una clase abstracta.

Representa la idea común de un menú.

Tiene:

```
protected Scanner scanner;
```

Y métodos comunes como:

```
leerEntero(String mensaje)
leerDouble(String mensaje)
leerTexto(String mensaje)
leerSiNo(String mensaje)
```

Estos métodos se reutilizan en los menús hijos.

28. Por qué `Menu` es abstracta

No tiene sentido crear un menú genérico sin opciones específicas.

Lo que sí tiene sentido es crear:

```
MenuArticulos
MenuCategorias
```

Por eso `Menu` define lo común, pero deja que las clases hijas implementen:

```
mostrarMenu()  
ejecutar()
```

Cada menú muestra opciones distintas y ejecuta acciones distintas.

29. Clase MenuArticulos

MenuArticulos hereda de Menu.

Su responsabilidad es administrar el CRUD de artículos.

Tiene acceso a dos repositorios:

```
private final Repositorio<Articulo> repositorioArticulos;  
private final Repositorio<Categoria> repositorioCategorias;
```

Necesita el repositorio de artículos para crear, listar, consultar, modificar y eliminar artículos.

Necesita el repositorio de categorías porque todo artículo debe asociarse a una categoría existente.

30. Métodos principales de MenuArticulos

La clase tiene métodos como:

```
ingresarArticulo()  
listarArticulos()  
consultarArticulo()  
modificarArticulo()  
eliminarArticulo()
```

Estos métodos implementan el CRUD.

También tiene métodos auxiliares:

```
pedirNombreArticulo()  
pedirPrecioArticulo()  
pedirGarantia()  
pedirDiasParaVencimiento()  
pedirCategoriaExistente()  
existeArticuloDuplicado(...)
```

Estos métodos hacen que el código sea más claro.

En vez de tener todo dentro de `ingresarArticulo()`, se separan pequeñas tareas.

31. Clase `MenuCategorias`

`MenuCategorias` también hereda de `Menu`.

Su responsabilidad es administrar el CRUD de categorías.

Tiene:

```
private final Repositorio<Categoria> repositorioCategorias;  
private final Repositorio<Articulo> repositorioArticulos;
```

Necesita el repositorio de categorías para crear, listar, consultar, modificar y eliminar categorías.

Necesita el repositorio de artículos para validar si una categoría tiene artículos asociados antes de eliminarla.

32. Validación importante en categorías

Antes de eliminar una categoría, el sistema debe verificar si está siendo usada.

Esto se resuelve con un método como:

```
categoriaTieneArticulosAsociados(Categoria categoria)
```

Esta validación protege la integridad del sistema.

Si se eliminara una categoría usada por artículos, esos artículos quedarían asociados a una categoría inexistente.

33. Paquete `utils`

El paquete `utils` contiene clases utilitarias.

En este proyecto son:

Validaciones
Secuencias

Estas clases no representan entidades del dominio.

No son artículos.

No son categorías.

Son clases de apoyo.

34. Clase Validaciones

Validaciones centraliza reglas comunes.

Tiene métodos como:

```
validarTextoNoVacio(String texto)
validarLongitudMaxima(String texto, int maximo)
validarNoNegativo(int valor)
validarNoNegativo(double valor)
```

Se declara como:

```
public final class Validaciones
```

Y tiene constructor privado:

```
private Validaciones() {
}
```

Esto indica que no se espera crear objetos de esta clase.

Se usa directamente así:

```
Validaciones.validarTextoNoVacio(nombre)
```

35. Clase Secuencias

Secuencias centraliza la generación automática de códigos.

Tiene:

```
private static int proximoCodigoArticulo = 1;  
private static int proximoCodigoCategoria = 1;
```

Y métodos:

```
generarCodigoArticulo()  
generarCodigoCategoria()
```

Esto evita pedirle al usuario que ingrese códigos manualmente.

También evita duplicación de lógica en distintos menús.

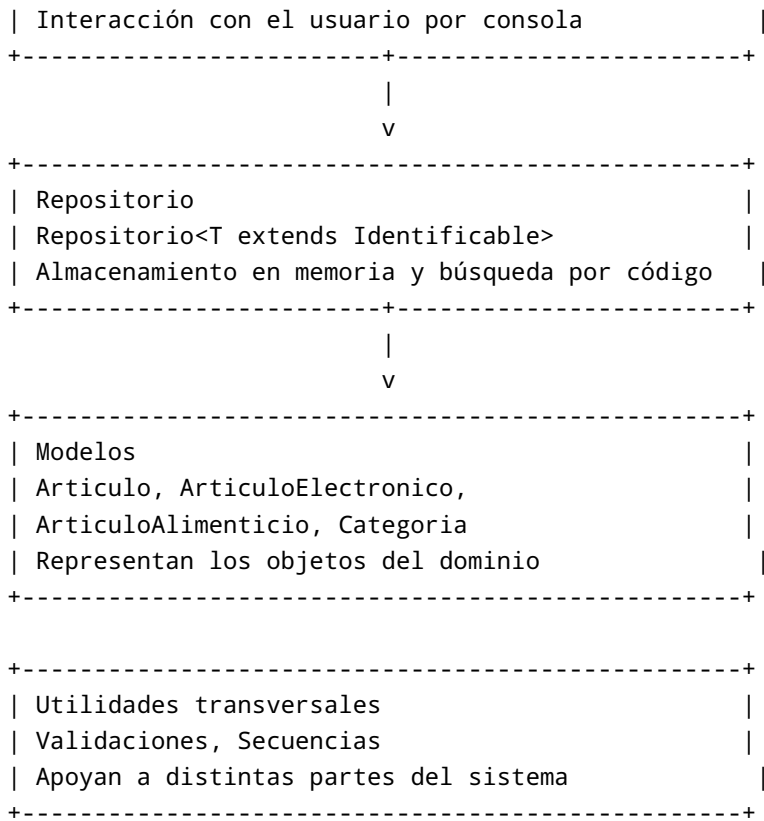
36. Flujo general de ejecución

El flujo general del programa es:

1. App inicia el sistema.
2. App crea Scanner.
3. App crea repositorio de categorías.
4. App crea repositorio de artículos.
5. App crea MenuCategorias.
6. App crea MenuArticulos.
7. App muestra menú principal.
8. El usuario elige artículos o categorías.
9. El menú correspondiente ejecuta su propio CRUD.
10. Los menús usan repositorios para guardar o buscar objetos.
11. Los modelos representan los datos del sistema.
12. Las utilidades ayudan con validaciones y códigos automáticos.

37. Arquitectura visual por responsabilidades

```
+-----+  
| App                                         |  
| Punto de entrada y coordinación general   |  
+-----+  
      |  
      v  
+-----+  
| Menús                                     |  
| Menu, MenuArticulos, MenuCategorias       |  
+-----+
```



38. Por qué esta arquitectura es adecuada para la preentrega

Esta arquitectura es adecuada porque el proyecto sigue siendo una aplicación de consola, pero ya no está todo mezclado.

El diseño permite mostrar varios conceptos importantes:

- separación de responsabilidades
- paquetes
- clases abstractas
- interfaces
- herencia
- polimorfismo
- generics
- repositorios
- clases utilitarias
- validaciones
- composición

Para un alumno principiante, es una buena transición entre:

programas simples con todo en main

Y:

proyectos backend más organizados

39. Por qué no es MVC puro

MVC significa:

Model - View - Controller

En una arquitectura MVC clásica:

- el modelo representa los datos
- la vista muestra información al usuario
- el controlador recibe acciones del usuario y coordina la lógica

Este proyecto tiene modelos claros:

Articulo
Categoria

Pero no tiene una capa `view` formal separada.

La visualización por consola está mezclada dentro de los menús.

Tampoco tiene controladores separados como en una aplicación web.

Por eso no conviene decir que es MVC puro.

Es más correcto decir que es una arquitectura modular por responsabilidades.

40. Comparación con MVC

Elemento del proyecto	Parecido en MVC	Aclaración
<code>Articulo</code> , <code>Categoria</code>	Model	Representan datos del dominio
<code>MenuArticulos</code> , <code>MenuCategorias</code>	View + Controller simplificado	Muestran opciones y ejecutan acciones

Elemento del proyecto	Parecido en MVC	Aclaración
<code>Repositorio</code>	Repository / acceso a datos	Guarda objetos en memoria
<code>App</code>	Bootstrap / inicio	Inicializa y coordina
<code>Validaciones</code> , <code>Secuencias</code>	Utilidades / helpers	Apoyan a varias clases

La comparación sirve para aprender, pero no debe confundirse con MVC completo.

41. Patrones y buenas prácticas presentes

Repository Pattern simplificado

La clase `Repositorio<T>` aplica una versión simple del patrón Repository.

Su objetivo es separar el almacenamiento de objetos del resto del sistema.

Los menús no necesitan saber cómo se guardan internamente los datos.

Solo llaman a métodos como:

```
agregar(...)
listar()
buscarPorCodigo(...)
eliminar(...)
```

Hoy el repositorio guarda en un `ArrayList`.

En un futuro podría guardar en una base de datos, y el resto del sistema no debería cambiar demasiado.

Template base con clase abstracta

`Menu` funciona como una base común para otros menús.

Define métodos comunes de lectura y obliga a implementar:

```
mostrarMenu()
ejecutar()
```

Esto permite que cada menú concreto respete una estructura general.

Utility Class

`Validaciones` y `Secuencias` son clases utilitarias.

Tienen:

- métodos estáticos
- constructor privado
- clase `final`

Esto muestra una práctica común para agrupar funciones auxiliares.

Polimorfismo

El sistema puede guardar distintos tipos de artículos en:

```
Repositorio<Articulo>
```

Ese repositorio puede contener:

```
ArticuloElectronico  
ArticuloAlimenticio
```

Esto es posible porque ambos heredan de `Articulo`.

Uso de interfaces

`Calculable` e `Identificable` permiten definir contratos.

`Calculable` dice que algo puede calcular precio final.

`Identificable` dice que algo tiene código.

Esto hace que el diseño sea más flexible.

42. Conceptos de POO aplicados

Encapsulamiento

Cada clase agrupa sus propios datos y métodos.

Ejemplo:

Categoria

tiene sus atributos privados y sus getters/setters.

Herencia

```
ArticuloElectronico extends Articulo
ArticuloAlimenticio extends Articulo
MenuArticulos extends Menu
MenuCategorias extends Menu
```

Polimorfismo

Una variable de tipo `Articulo` puede apuntar a un `ArticuloElectronico` o a un `ArticuloAlimenticio`.

Abstracción

`Articulo` y `Menu` son abstractas porque representan conceptos generales.

Interfaces

`Calculable` e `Identificable` definen obligaciones.

Composición

`Articulo` tiene una `Categoria`.

`MenuArticulos` tiene repositorios.

`MenuCategorias` tiene repositorios.

43. Qué está muy bien logrado en el proyecto

El proyecto tiene una estructura clara para nivel inicial/intermedio.

Está bien logrado porque:

- no deja todo dentro de `main`
- separa modelos, menús, repositorio y utilidades
- usa herencia con sentido
- usa interfaces con propósito concreto
- usa generics de manera útil
- evita duplicar un repositorio para cada entidad

- usa validaciones centralizadas
- genera códigos automáticamente
- protege parte del acceso a datos usando repositorio
- permite crecer agregando nuevos tipos de artículos

44. Qué se podría mejorar en una versión futura

Como proyecto didáctico está bien planteado.

Pero en una evolución más profesional se podría mejorar:

Crear una capa de servicios

Actualmente los menús tienen bastante lógica de negocio.

Por ejemplo:

- validar duplicados
- validar eliminación de categorías
- crear artículos
- modificar datos

En una arquitectura más avanzada, parte de esa lógica podría ir a servicios:

```
ArticuloService  
CategoriaService
```

Separar vista de lógica

Los menús muestran texto y también ejecutan lógica.

En una arquitectura más estricta se podría separar:

- presentación por consola
- lógica de aplicación

Persistencia real

Actualmente los datos viven en memoria.

Cuando el programa termina, se pierden.

En una versión futura se podría guardar en:

- archivos
- base de datos

- API REST

Manejo de errores más formal

Actualmente se usan mensajes por consola.

Más adelante podrían usarse excepciones personalizadas.

45. Cómo se conectaría esto con Spring Boot

Este proyecto prepara muy bien el camino hacia Spring Boot.

La equivalencia conceptual sería:

Proyecto consola	Spring Boot
<code>App</code>	Clase principal con <code>@SpringBootApplication</code>
<code>MenuArticulos</code>	Controller o capa de entrada
<code>Repositorio<T></code>	Repository
<code>Articulo</code> , <code>Categoria</code>	Entity / Model
<code>Validaciones</code>	Validadores / reglas auxiliares
Lógica en menús	Service

La diferencia es que en Spring Boot las entradas no vienen por consola.

Vienen por HTTP, por ejemplo:

```
POST /articulos
GET /categorias
PUT /articulos/{id}
DELETE /categorias/{id}
```

Pero la idea de separar responsabilidades es la misma.

46. Resumen final

El proyecto de preentrega implementa una arquitectura modular para Java consola.

No es un MVC puro.

No es todavía una arquitectura completa de backend.

Pero sí es una estructura muy adecuada para enseñar buenas prácticas iniciales.

La arquitectura separa:

- inicio de aplicación: `App`
- interacción con usuario: `Menu`, `MenuArticulos`, `MenuCategorias`
- modelos del dominio: `Articulo`, `ArticuloElectronico`, `ArticuloAlimenticio`, `Categoria`
- contratos: `Calculable`, `Identificable`
- almacenamiento en memoria: `Repositorio<T>`
- utilidades: `Validaciones`, `Secuencias`

El valor principal del proyecto es que muestra cómo un CRUD puede dejar de ser un programa monolítico y empezar a organizarse como un sistema más mantenible, extensible y profesional.

Para alumnos principiantes, este proyecto permite ver de forma concreta cómo se aplican conceptos que muchas veces parecen abstractos:

- herencia
- polimorfismo
- clases abstractas
- interfaces
- generics
- composición
- encapsulamiento
- separación de responsabilidades

La arquitectura elegida es correcta para el objetivo pedagógico: enseñar POO en Java mediante un CRUD realista, pero sin agregar todavía la complejidad de bases de datos, frameworks o APIs web.